Friendly Remediation Report for Saltillo

> *Plain-language report for readers who do not work in technical or security roles.*

Quick summary

Saltillo shows a critical level of risk with score 83/100 based on 6 main issue(s). This version explains the situation in simpler language, keeps the uncertainty visible, and highlights the most important next actions first.

Basic context

- Target: Saltillo
- Public ID: mx-coahuila-saltillo
- Audience: Friendly / nontechnical
- Generated at: 2026-01-01T00:00:00.000Z
- Generated by: deterministic-fallback

Most important next actions

1. Priority 1: Confirm the WordPress core and plugin patch level, then upgrade to the latest supported release. Why it matters: wordpress 6.4; confidence 0.8; references CVE-2024-31210.
2. Priority 2: Enable forced secure browsing rule after confirming HTTPS is stable for the site and subdomains. Why it matters: strict-transport-security was not present in the selected response headers.
3. Priority 3: Add a tested Content-Security-Policy that limits script, frame, and object sources. Why it matters: content-security-policy was not present in the selected response headers.
4. Priority 4: Send browser setting that blocks unsafe file-type guessing: nosniff on HTML and static asset responses. Why it matters: browser setting that blocks unsafe file-type guessing was not present in the selected response headers.
5. Priority 5: Restrict administrative entry points with access controls, MFA, and monitoring where operationally possible. Why it matters: /wp-login.php returned 200.

What this report covers

Saltillo was treated as the scoped public-facing target for this report. The report generator normalized the available structured input and limited the output to the provided target context.

- Primary public URL: https://www.saltillo.gob.mx

- Region or state reference: Coahuila

What was allowed

No explicit authorization object was supplied. The report generator treated the input as evidence-only and did not add new tests or claims.

- No detailed authorization fields were supplied in the current input payload.

How the review was done

The report was assembled from structured inputs, normalized into a consistent evidence model, and then rendered into audience-specific fix guidance.

- Reviewed structured scan evidence for https://www.saltillo.gob.mx.
- Normalized flexible structured input into a consistent report contract before generating the PDFs.
- Preserved only evidence-backed observations and did not add new findings without source support.
- Excluded exploit payloads, credential use, and operational attack instructions from the output.

What needs attention

These are the main issues that deserve attention first. The list is ordered by likely impact and by how clear the available evidence is.

- high: 1
- medium: 4
- low: 1
- Top issue: WordPress 6.4 requires patch review
- Top issue: Missing browser rule that forces secure connections
- Top issue: Missing browser rule that limits what the site can load

Issue-by-issue explanation

1. WordPress 6.4 requires patch review

- Severity: high
- Status: observed
- Confidence: high
- Category: known-vulnerability

Description

The scanner observed a WordPress 6.4 generator string. This MVP treats the version as a known-risk demo match when confidence is high, without claiming exploitation.

Evidence

wordpress 6.4; confidence 0.8; references CVE-2024-31210.

Recommended remediation

Confirm the WordPress core and plugin patch level, then upgrade to the latest supported release.

Remediation steps

- Confirm the WordPress core and plugin patch level, then upgrade to the latest supported release.
- Assign an owner and due date before treating the item as resolved.

Verification steps

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for wordpress 6.4 requires patch review and compare it with the current evidence.

2. Missing browser rule that forces secure connections

- Severity: medium
- Status: observed
- Confidence: medium
- Category: headers

Description

A baseline browser security response header was not observed in the safe public HTTP response.

Evidence

strict-transport-security was not present in the selected response headers.

Recommended remediation

Enable forced secure browsing rule after confirming HTTPS is stable for the site and subdomains.

Remediation steps

- Enable forced secure browsing rule after confirming HTTPS is stable for the site and subdomains.
- Assign an owner and due date before treating the item as resolved.

Verification steps

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for missing browser rule that forces secure connections and compare it with the current evidence.

3. Missing browser rule that limits what the site can load

- Severity: medium
- Status: observed
- Confidence: medium

- Category: headers

Description

A baseline browser security response header was not observed in the safe public HTTP response.

Evidence

content-security-policy was not present in the selected response headers.

Recommended remediation

Add a tested Content-Security-Policy that limits script, frame, and object sources.

Remediation steps

- Add a tested Content-Security-Policy that limits script, frame, and object sources.
- Assign an owner and due date before treating the item as resolved.

Verification steps

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for missing browser rule that limits what the site can load and compare it with the current evidence.

4. Missing browser setting that blocks unsafe file-type guessing

- Severity: medium
- Status: observed
- Confidence: medium
- Category: headers

Description

A baseline browser security response header was not observed in the safe public HTTP response.

Evidence

browser setting that blocks unsafe file-type guessing was not present in the selected response headers.

Recommended remediation

Send browser setting that blocks unsafe file-type guessing: nosniff on HTML and static asset responses.

Remediation steps

- Send browser setting that blocks unsafe file-type guessing: nosniff on HTML and static asset responses.
- Assign an owner and due date before treating the item as resolved.

Verification steps

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for missing browser setting that blocks unsafe file-type guessing and compare it with the current evidence.

5. Public admin path is reachable

- Severity: medium
- Status: observed
- Confidence: medium
- Category: admin-exposure

Description

One or more common website management software or generic admin paths responded successfully to safe safe public checks.

Evidence

/wp-login.php returned 200.

Recommended remediation

Restrict administrative entry points with access controls, MFA, and monitoring where operationally possible.

Remediation steps

- Restrict administrative entry points with access controls, MFA, and monitoring where operationally possible.
- Assign an owner and due date before treating the item as resolved.

Verification steps

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for public admin path is reachable and compare it with the current evidence.

6. website management software fingerprint was detected

- Severity: low
- Status: observed
- Confidence: low
- Category: cms

Description

Public page evidence revealed a likely website management software fingerprint. This is informational unless combined with other risk evidence.

Evidence

wordpress 6.4; confidence 0.8; evidence generator:WordPress 6.4, marker:wordpress.

Recommended remediation

Keep website management software core, extensions, and themes patched, and remove unnecessary public version disclosures where practical.

Remediation steps

- Keep website management software core, extensions, and themes patched, and remove unnecessary public version disclosures where practical.
- Assign an owner and due date before treating the item as resolved.

Verification steps

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for website management software fingerprint was detected and compare it with the current evidence.

What was not tested

The current material does not cover every possible check, and the following activities were intentionally left out or not supplied.

- Active exploitation, credential testing, fuzzing, destructive requests, and private-network actions were out of scope.

What the current evidence confirms

These notes explain what the current evidence does and does not confirm.

- The safe public target appeared reachable during the supplied observation window.
- Observed HTTP status: 200.
- No explicit controlled validation result was supplied in the structured input.

Important limits

These limits matter because they explain what still needs human follow-up before making a final decision.

- Authorization scope details were not fully supplied.
- No explicit validation result set was supplied, so unresolved uncertainty remains.
- The report summarizes the provided evidence and should not be interpreted as proof of how easily the issue could be misused or breach.

Recommended next actions

These are the clearest next steps to reduce the risk in a practical way.

- Confirm the WordPress core and plugin patch level, then upgrade to the latest supported release.
- Assign an owner and due date before treating the item as resolved.
- Confirm ownership for wordpress 6.4 requires patch review before the next review cycle.
- Enable forced secure browsing rule after confirming HTTPS is stable for the site and subdomains.
- Confirm ownership for missing browser rule that forces secure connections before the next review cycle.
- Add a tested Content-Security-Policy that limits script, frame, and object sources.
- Confirm ownership for missing browser rule that limits what the site can load before the next review cycle.

- Send browser setting that blocks unsafe file-type guessing: nosniff on HTML and static asset responses.
- Confirm ownership for missing browser setting that blocks unsafe file-type guessing before the next review cycle.
- Restrict administrative entry points with access controls, MFA, and monitoring where operationally possible.

How to check the fixes

After each fix, use a simple follow-up review to confirm the public signs improved.

- Re-run the same bounded check after the mitigation is applied.
- Record the post-change result for wordpress 6.4 requires patch review and compare it with the current evidence.
- Record the post-change result for missing browser rule that forces secure connections and compare it with the current evidence.
- Record the post-change result for missing browser rule that limits what the site can load and compare it with the current evidence.
- Record the post-change result for missing browser setting that blocks unsafe file-type guessing and compare it with the current evidence.
- Record the post-change result for public admin path is reachable and compare it with the current evidence.
- Record the post-change result for website management software fingerprint was detected and compare it with the current evidence.